And now here is `lines`:

```
def lines(filename: String): Process[IO,String] =
  resource
    { IO(io.Source.fromFile(filename)) }
    { src =>
        lazy val iter = src.getLines // a stateful iterator
        def step = if (iter.hasNext) Some(iter.next) else None
        lazy val lines: Process[IO,String] = eval(IO(step)).flatMap {
          case None => Halt(End)
          case Some(line) => Emit(line, lines)
        }
        lines
    }
    { src => eval_ { IO(src.close) } }
```

The `resource` combinator, using `onComplete`, ensures that our underlying resource is freed regardless of how the process is terminated. The only thing we need to ensure is that `|>` and other consumers of `lines` gracefully terminate it when they're finished consuming. We'll address this next, when we redefine single-input processes and implement the `|>` combinator for our generalized `Process` type.

15.3.3 Single-input processes

We now have nice, resource-safe sources, but we don't yet have any way to apply transformations to them. Fortunately, our `Process` type can also represent the single-input processes we introduced earlier in this chapter. To represent `Process1[I,O]`, we craft an appropriate `F` that only allows the `Process` to make requests for elements of type `I`. Let's look at how this works—the encoding is unusual in Scala, but there's nothing fundamentally new here:

```
case class Is[I]() {
  sealed trait f[X]
  val Get = new f[I] {}
}
```

It's strange to define the trait `f` inside of `Is`. Let's unpack what's going on. Note that `f` takes one parameter, `X`, but we have just one instance, `Get`, which fixes `X` to be the `I` in the outer `Is[I]`. Therefore, the type `Is[I]#f`[8] can only ever be a request for a value of type `I`! Given the type `Is[I]#f[A]`, Scala will complain with a type error unless the type `A` *is* the type `I`. Now that we have all this, we can define `Process1` as just a type alias:

```
type Process1[I,O] = Process[Is[I]#f, O]
```

To see what's going on, it helps to substitute the definition of `Is[I]#f` into a call to `Await`:

```
case class Await[A,O](
  req: Is[I]#f[A], recv: Either[Throwable,A] => Process[Is[I]#f,O]
) extends Process[Is[I]#f,R]
```

[8] Note on syntax: recall that if `x` is a type, `x#foo` references the type `foo` defined inside `x`.

From the definition of `Is[I]#f`, we can see that `req` can only be `Get: f[I]`, the only value of this type. Therefore, `I` and `A` must be the same type, so `recv` must accept an `I` as its argument, which means that `Await` can only be used to request `I` values. This is important to understand—if this explanation didn't make sense, try working through these definitions on paper, substituting the type definitions.

Our `Process1` alias supports all the same operations as our old single-input `Process`. Let's look at a couple. We first introduce a few helper functions to improve type inference when calling the `Process` constructors.

Listing 15.7 Type inference helper functions

```
def await1[I,O](
    recv: I => Process1[I,O],
    fallback: Process1[I,O] = halt1[I,O]): Process1[I, O] =
  Await(Get[I], (e: Either[Throwable,I]) => e match {
    case Left(End) => fallback
    case Left(err) => Halt(err)
    case Right(i) => Try(recv(i))
  })

def emit1[I,O](h: O, tl: Process1[I,O] = halt1[I,O]): Process1[I,O] =
  emit(h, tl)

def halt1[I,O]: Process1[I,O] = Halt[Is[I]#f, O](End)
```

Using these, our definitions of combinators like `lift` and `filter` look almost identical to before, except they return a `Process1`:

```
def lift[I,O](f: I => O): Process1[I,O] =
  await1[I,O](i => emit(f(i))) repeat

def filter[I](f: I => Boolean): Process1[I,I] =
  await1[I,I](i => if (f(i)) emit(i) else halt1) repeat
```

Let's look at process composition next. The implementation looks similar to before, but we make sure to run the latest `cleanup` action of the left process before the right process halts:

```
def |>[O2](p2: Process1[O,O2]): Process[F,O2] = {
  p2 match {
    case Halt(e) => this.kill onHalt { Halt(e) ++ Halt(e2) }
    case Emit(h, t) => Emit(h, this |> t)
    case Await(req,recv) => this match {
      case Halt(err) => Halt(err) |> recv(Left(err))
      case Emit(h,t) => t |> Try(recv(Right(h)))
      case Await(req0,recv0) => await(req0)(recv0 andThen (_ |> p2))
    }
  }
}
def pipe[O2](p2: Process1[O,O2]): Process[F,O2] =
  this |> p2
```

If `this` has halted, do the appropriate cleanup.

Before halting, gracefully terminate `this`, using `++` to preserve the first error, if any occurred.